

EX: 01 - Using Wireshark to Explore Protocol Headers

Aim

To use Wireshark to capture network packets, observe different protocol headers, and analyze packet-related values such as frame size, header overhead, data rate, and packet filtering/blocking percentage.

How to Use the Tool

1. Open **Wireshark** from the Start Menu.
2. Select the active network interface such as **Wi-Fi** or **Ethernet**.
3. Click the **Start Capture** button or double-click the selected interface.
4. Open a web browser and visit any website to generate network traffic.
5. Return to Wireshark and click any captured packet.
6. Expand the packet details section and observe **Ethernet II**, **Internet Protocol**, **TCP**, and **Payload** fields.
7. Note the required values such as header size, packet size, number of packets, and time duration.
8. Click the **Stop Capture** button after completing the observation.

Procedure

1. Install Wireshark on the system and open it with proper permission. Select the active network interface such as Wi-Fi or Ethernet for packet capturing.
2. Start packet capture on the selected interface. Ensure that the system is connected to the network so that live packets can be captured.
3. Generate normal network traffic by opening a website or using any network-based application. This helps Wireshark capture packets from different protocols.
4. Observe the captured packets in the packet list window. Select any packet and expand the packet details section to view protocol information.
5. Identify the Ethernet header, IP header, TCP header, and payload section from the selected packet. Note important fields such as source MAC, destination MAC, source IP, destination IP, port number, and protocol type.
6. Record the required values such as header size, payload size, packet size, number of packets, and time duration. These values are used for frame size, overhead, and data rate calculations.

7. Perform the required calculations for Q1, Q11, and Q21 using the given values.
Calculate total frame size, header overhead, average data rate, and blocking percentage.
8. Stop the packet capture after collecting enough packets. Record the calculated output clearly and write the final result.

Program / Calculation

Q1

Question:

A captured frame consists of Ethernet header = 14 bytes, IP header = 20 bytes, TCP header = 24 bytes, and payload = 1200 bytes. Calculate the total frame size and percentage of header overhead.

Given:

Ethernet header = 14 bytes

IP header = 20 bytes

TCP header = 24 bytes

Payload = 1200 bytes

Calculation:

Total header size = Ethernet + IP + TCP
= 14 + 20 + 24
= 58 bytes

Total frame size = Header size + Payload
= 58 + 1200
= 1258 bytes

Header overhead percentage = Header size / Total frame size × 100
= 58 / 1258 × 100
= 4.61%

Output:

Total frame size = 1258 bytes
Header overhead = 4.61%

Q11

Question:

During packet capture, 2500 packets of size 1518 bytes are observed over 5 seconds. Compute the average data rate in Mbps and analyze whether the network is congested.

Given:

Number of packets = 2500

Size of each packet = 1518 bytes

Time = 5 seconds

Calculation:

Total data in bytes = Number of packets × Packet size

= 2500 × 1518

= 3,795,000 bytes

Total data in bits = 3,795,000 × 8

= 30,360,000 bits

Average data rate = Total bits / Time

= 30,360,000 / 5

= 6,072,000 bps

Average data rate in Mbps = 6,072,000 / 1,000,000

= 6.072 Mbps

Output:

Average data rate = 6.072 Mbps

Network condition:

If the network capacity is 100 Mbps or 1 Gbps, the network is not congested. If the network capacity is 10 Mbps, the traffic is moderate.

Q21

Question:

A firewall processes 50,000 packets, out of which 3,500 are blocked. Calculate the blocking percentage and analyze whether the firewall policy is strict enough.

Given:

Total packets processed = 50,000

Blocked packets = 3,500

Calculation:

Blocking percentage = Blocked packets / Total packets × 100

= 3500 / 50000 × 100

= 7%

Output:

Blocking percentage = 7%

Policy note:

The firewall blocked only 7% of packets. The policy may be moderately strict, but its effectiveness depends on whether the blocked packets were malicious.

Result

Thus, network packets were analyzed using the concept of Wireshark protocol header observation. The total frame size, header overhead, average data rate, and packet blocking percentage were calculated successfully.

EX: 02 - Demonstrate Two Different Certificates Producing Same MD5 Hash

Aim

To demonstrate the weakness of MD5 hashing by studying hash collisions and understanding how two different inputs or certificates can produce the same MD5 hash value.

How to Use the Tool

1. Create a folder in the system, for example:

C:\NSLab\MD5

2. Save the two sample certificate files inside the folder.
3. Open **Command Prompt**.
4. Move to the certificate folder:

cd C:\NSLab\MD5

5. Calculate the MD5 hash of the first certificate using certutil:

certutil -hashfile certificate1.crt MD5

6. Calculate the MD5 hash of the second certificate:

certutil -hashfile certificate2.crt MD5

7. Compare both MD5 hash values displayed in the command prompt.
8. If both hash values are the same, the MD5 collision is verified.

Procedure

1. Study the concept of cryptographic hashing and understand how MD5 produces a fixed 128-bit hash value for any input data.
2. Understand the meaning of hash collision. A collision occurs when two different inputs produce the same hash value.
3. Use sample certificate files or already available MD5 collision examples for safe laboratory study. The files should be used only for academic and demonstration purposes.
4. Use Windows Command Prompt and the certutil command to calculate the MD5 hash value of the given sample files.
5. Compare both hash values carefully. If both different files produce the same MD5 hash, the collision is verified.
6. Calculate the total number of possible MD5 hash values using 2^{128} .
7. Estimate collision attempts using the birthday paradox method.

8. Study the effect of MD5 collision on certificate authentication and digital signatures. Understand why MD5 is not used in modern secure systems.
9. Record the calculated values, hash comparison output, and final result.

Program / Calculation

Q2

Given:

MD5 hash size = 128 bits

Total possible hashes:

$$2^{128} = 3.40 \times 10^{38}$$

Collision attempts using birthday paradox:

$$2^{64} = 1.84 \times 10^{19}$$

Final Answer:

$$\text{Total possible hashes} = 2^{128} = 3.40 \times 10^{38}$$

$$\text{Attempts required for collision} \approx 2^{64} = 1.84 \times 10^{19}$$

Q12

Given:

Two different certificates produce identical MD5 hashes.

MD5 output size = 128 bits

Random collision probability for one pair:

$$1 / 2^{128}$$

Birthday collision attempt:

$$2^{64}$$

Final Answer:

$$\text{Collision probability for one random pair} = 1 / 2^{128}$$

$$\text{Birthday collision attempt} \approx 2^{64}$$

Impact:

MD5 collision can affect authentication systems because a fake certificate may produce the same hash as a valid certificate.

Q22

Given:

Total employees = 800

Employees clicked malicious link = 120

Phishing success rate:

$$= 120 / 800 \times 100$$

$$= 15\%$$

Final Answer:

Phishing success rate = 15%

Organization awareness level:

The security awareness level is moderate and needs improvement through proper phishing awareness training.

Result

Thus, MD5 hash collision and its security impact were studied successfully.

EX: 03 - Computing MACs, HASH and HMAC for Messages

Aim

To compute HASH and HMAC values for messages and understand their use in message integrity and authentication.

How to Use the Tool

1. Open **Python IDLE** or **Jupyter Notebook**.
2. Create a new Python file or notebook.
3. Import the required modules:

```
import hashlib
```

```
import hmac
```

4. Define a message and secret key:

```
msg = b"Hello"
```

```
key = b"secret"
```

5. Generate SHA-256 hash:

```
hash_value = hashlib.sha256(msg).hexdigest()
```

6. Generate HMAC using SHA-256:

```
hmac_value = hmac.new(key, msg, hashlib.sha256).hexdigest()
```

7. Print both output values:

```
print("SHA-256:", hash_value)
```

```
print("HMAC:", hmac_value)
```

8. Run the program and note the HASH and HMAC values.

Procedure

1. Open Python IDLE, command prompt, or Jupyter Notebook in the system. Make sure Python is installed and working properly.
2. Import the required Python modules such as hashlib and hmac. The hashlib module is used for hashing, and the hmac module is used for keyed hashing.
3. Define a sample message and a secret key. The message is used to generate the hash value, and the key is used to generate the HMAC value.
4. Compute the SHA-256 hash value for the message. This hash value helps verify whether the message content has been changed.

5. Compute the HMAC value using the secret key and SHA-256 algorithm. HMAC provides both message integrity and authentication.
6. Observe the output values of HASH and HMAC. Compare the outputs and understand that HMAC is more secure because it uses a secret key.
7. Use the given values to calculate total input size, output size, computational overhead, and malicious request percentage.
8. Record the calculated values, program output, and final result.

Program / Calculation

Q3

Given:

Message size = 2048 bits

Key size = 128 bits

SHA-256 output size = 256 bits

Total input size:

= Message size + Key size

= 2048 + 128

= 2176 bits

SHA-256 output = 256 bits

HMAC-SHA256 output = 256 bits

Combined output:

= 256 + 256

= 512 bits

Final Answer:

Total input size = 2176 bits

SHA-256 output = 256 bits

HMAC output = 256 bits

Combined output = 512 bits

Q13

Given:

Number of messages = 10,000

For HASH:

Hash operations = 10,000

For HMAC:

HMAC uses inner and outer hashing.

Approximate HMAC operations:

$$= 2 \times 10,000$$

$$= 20,000 \text{ operations}$$

Final Answer:

$$\text{HASH operations} = 10,000$$

$$\text{HMAC operations} \approx 20,000$$

Trade-off:

HASH is faster, but HMAC is more secure because it uses a secret key.

Q23

Given:

$$\text{Total requests} = 2,000,000$$

$$\text{Legitimate requests} = 1,700,000$$

Malicious requests:

$$= \text{Total requests} - \text{Legitimate requests}$$

$$= 2,000,000 - 1,700,000$$

$$= 300,000$$

Malicious request percentage:

$$= 300,000 / 2,000,000 \times 100$$

$$= 15\%$$

Final Answer:

$$\text{Malicious requests} = 300,000$$

$$\text{Malicious request percentage} = 15\%$$

Possible DDoS indicator:

A high number of malicious requests may indicate abnormal traffic or a possible DDoS attack.

Result

Thus, HASH and HMAC calculations were performed successfully.

EX: 04 - Implement and Demonstrate Buffer Overflow Attack

Aim

To understand buffer overflow in a controlled lab environment and calculate the effect of excess input on memory.

How to Use the Tool

1. Open a safe local lab environment or use the given theoretical values.
2. Identify the buffer size from the question.
3. Identify the input size from the question.
4. Use the formula:

Overflow size = Input size – Buffer size

5. Check whether the input size crosses the return address position.
6. If the input size is greater than the return address position, the return address is affected.
7. Record the overflow size and memory impact.
8. This experiment should be done only for academic understanding in a controlled lab environment.

Procedure

1. Study the concept of buffer overflow in a controlled lab environment. Understand that buffer overflow occurs when input data is larger than the allocated memory buffer.
2. Identify the buffer size and input size from the given question. These values are required to calculate whether overflow occurs.
3. Compare the input size with the buffer size. If the input size is greater than the buffer size, the extra data causes overflow.
4. Calculate the overflow size by subtracting the buffer size from the input size. This shows how many bytes exceed the allocated memory.
5. Check whether the excess input reaches important memory areas such as the return address.
6. If the input crosses the return address position, the program control flow may be affected.
7. Observe possible effects such as program crash, abnormal output, or memory overwrite in a safe lab environment.

8. Study security risks caused by buffer overflow, such as memory corruption, unauthorized code execution, and system compromise.
9. Record the overflow size, affected memory condition, security risk, and final result.

Program / Calculation

Q4

Concept:

Buffer overflow occurs when the input data is larger than the allocated buffer size.

Example:

If a buffer can store only 100 bytes and the user enters more than 100 bytes, the extra data may overwrite nearby memory.

Impact:

Program crash

Memory corruption

Unauthorized code execution

System compromise

Final Answer:

Buffer overflow affects system security by overwriting memory and causing abnormal program behavior.

Q14

Given:

Buffer size = 100 bytes

Input size = 180 bytes

Return address position = 120 bytes

Overflow size:

= Input size – Buffer size

= 180 – 100

= 80 bytes

Return address check:

Input size = 180 bytes

Return address position = 120 bytes

Since $180 > 120$, return address is affected.

Final Answer:

Overflow size = 80 bytes

Return address is affected

Q24

Given:

Password testing speed = 500,000 passwords/second

Total passwords = 1,000,000,000

Time required:

$$= 1,000,000,000 / 500,000$$

$$= 2000 \text{ seconds}$$

Convert into minutes:

$$= 2000 / 60$$

$$= 33.33 \text{ minutes}$$

Final Answer:

Time required = 2000 seconds

Time required = 33 minutes 20 seconds

Password security risk:

Weak passwords can be cracked quickly. Strong passwords and multi-factor authentication improve security.

Result

Thus, buffer overflow and related security risks were studied successfully.

EX: 05 - Implement and Demonstrate DoS and DDoS Attacks

Aim

To understand the effect of DoS and DDoS attacks on server availability and calculate overload, dropped requests, and detection accuracy.

How to Use the Tool

1. Use only the given traffic values or a safe lab simulation.
2. Note the normal server capacity from the question.
3. Note the incoming attack traffic value from the question.
4. Calculate excess traffic using:

Excess traffic = Attack traffic – Server capacity

5. Calculate overload percentage using:

Overload percentage = Excess traffic / Server capacity × 100

6. Calculate dropped requests using:

Dropped requests = Attack traffic – Server capacity

7. Study the effect of excessive traffic on availability.
8. Do not generate DoS or DDoS traffic on public websites, college networks, or unauthorized systems.

Procedure

1. Study the concept of Denial of Service and Distributed Denial of Service attacks. Understand that these attacks affect service availability by sending excessive traffic.
2. Use only a controlled lab or given traffic values for this experiment. Do not perform DoS or DDoS testing on public systems or real networks.
3. Identify the normal server capacity and incoming attack traffic rate from the question. These values are used to calculate overload.
4. Calculate the excess traffic by subtracting server capacity from incoming attack traffic.
5. Calculate the overload percentage using the excess traffic and normal server capacity.
6. Calculate dropped requests by finding the difference between attack traffic and server capacity.
7. Study the effect of excessive traffic on system availability, such as slow response, timeout, server crash, and service unavailability.
8. Study mitigation techniques such as rate limiting, firewall filtering, IDS/IPS monitoring, load balancing, CDN usage, DDoS protection, and blocking suspicious IP addresses.

- Record the overload percentage, dropped requests, detection accuracy, mitigation methods, and final result.

Program / Calculation

Q5

Given:

Server capacity = 3000 requests/second

Attack traffic = 9000 requests/second

Excess traffic:

= Attack traffic – Server capacity

= 9000 – 3000

= 6000 requests/second

Overload percentage:

= Excess traffic / Server capacity × 100

= 6000 / 3000 × 100

= 200%

Dropped requests:

= 9000 – 3000

= 6000 requests/second

Final Answer:

Overload percentage = 200%

Dropped requests = 6000 requests/second

Q15

Excessive traffic affects system availability by causing:

Slow response

Request timeout

High CPU usage

High memory usage

Bandwidth exhaustion

Server crash

Service unavailability

Mitigation techniques:

Rate limiting

Firewall filtering

IDS/IPS monitoring

Load balancing

CDN usage

DDoS protection

Blocking suspicious IPs

Final Answer:

Excessive traffic reduces service availability. It can be controlled using filtering, rate limiting, and DDoS protection.

Q25

Given:

Actual attacks = 1000

Detected attacks = 850

Detection accuracy:

= Detected attacks / Actual attacks × 100

= 850 / 1000 × 100

= 85%

Missed attacks:

= 1000 – 850

= 150

Final Answer:

Detection accuracy = 85%

Missed attacks = 150

Result

Thus, DoS and DDoS attack impact was studied successfully.

EX: 07 - Implement Botnet Attack Detection Using Public Dataset

Aim

To detect botnet or malicious activity using packet data and calculate detection accuracy, false negative rate, and attack impact.

How to Use the Tool

1. Open **Anaconda Navigator**.
2. Launch **Jupyter Notebook**.
3. Create a new Python notebook.
4. Import the required libraries:

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report
```

5. Load the dataset:

```
data = pd.read_csv("dataset.csv")
```

6. Separate input features and output labels.
7. Split the dataset into training and testing data.
8. Train the machine learning model and evaluate the output.

Procedure

1. Install Python, Jupyter Notebook, Pandas, NumPy, and Scikit-learn in the system. These tools are used for loading, processing, and evaluating the dataset.
2. Download or use a public botnet dataset for laboratory study. The dataset should contain normal and malicious traffic records.
3. Load the dataset using Pandas and observe the number of records, features, and class labels. This helps understand the structure of the dataset.
4. Preprocess the dataset by checking missing values, removing unwanted columns, and separating features and labels.
5. Split the dataset into training and testing data. The training data is used to train the model, and the testing data is used to evaluate performance.

6. Train a machine learning classifier such as Random Forest to identify malicious or botnet traffic.
7. Evaluate the model using values such as detected malicious packets, false negatives, accuracy, precision, and recall.
8. Study compromised system detection techniques such as IDS alerts, SIEM logs, suspicious DNS requests, unusual outbound traffic, and endpoint alerts.
9. Record the detection accuracy, false negative rate, ransomware impact calculation, and final result.

Program / Calculation

Q7

Given:

Total packets = 100,000

Malicious packets = 20,000

Correctly detected malicious packets = 18,000

Detection accuracy:

$$= 18,000 / 20,000 \times 100$$

$$= 90\%$$

False negatives:

$$= \text{Malicious packets} - \text{Correctly detected malicious packets}$$

$$= 20,000 - 18,000$$

$$= 2,000$$

False negative rate:

$$= 2,000 / 20,000 \times 100$$

$$= 10\%$$

Final Answer:

Detection accuracy = 90%

False negative rate = 10%

Q17

Techniques used to detect compromised systems:

IDS/IPS alerts

SIEM log analysis

Unusual outbound traffic monitoring

Suspicious DNS request detection

Antivirus or EDR alerts

Unexpected login attempts

Unknown running processes
High CPU or network usage
Connection to malicious IP addresses
File integrity monitoring

Final Answer:

Compromised systems can be detected using network monitoring, endpoint monitoring, and log analysis.

Q27

Given:

Encrypted data = 75 GB

Total data = 120 GB

Encrypted data percentage:

$$= 75 / 120 \times 100$$

$$= 62.5\%$$

Final Answer:

Encrypted data percentage = 62.5%

Business impact:

The business impact is high because more than half of the data is encrypted.

Result

Thus, botnet detection and compromised system detection were studied successfully.

EX: 08 - Explore and Install Snort Intrusion Detection Tool

Aim

To explore Snort IDS and calculate packet processing count, alert percentage, and login failure percentage.

How to Use the Tool

1. Install **Npcap** first because Snort needs packet capture support.
2. Install **Snort** in the system.
3. Open **Command Prompt as Administrator**.
4. Move to the Snort binary folder:

```
cd C:\Snort\bin
```

5. List available network interfaces:

```
snort -W
```

6. Test the Snort configuration file:

```
snort -T -c C:\Snort\etc\snort.conf
```

7. Start Snort monitoring:

```
snort -A console -q -c C:\Snort\etc\snort.conf -i 1
```

8. Replace 1 with the correct interface number shown by snort -W.

Procedure

1. Install Npcap or WinPcap in the system. This allows Snort to capture packets from the network interface.
2. Download and install Snort IDS for the operating system used in the lab. Verify that the Snort folder contains configuration, rule, and log directories.
3. Configure the Snort installation path and check the snort.conf file. Ensure that rule paths and network variables are properly configured.
4. Run Snort in test mode to verify whether the configuration file has any errors. If errors are found, correct the rule path or configuration settings.
5. Start Snort packet monitoring on the selected network interface. Allow Snort to inspect network traffic.
6. Observe packets and alerts generated by Snort during monitoring. The alert output helps identify suspicious activity.
7. Use the given question values to calculate total packets analyzed, alert percentage, and login failure percentage.

8. Record the Snort output, calculated values, possible brute-force signs, and final result.

Program / Calculation

Q8

Given:

Snort processing speed = 10,000 packets/second

Time = 15 minutes

Convert time:

15 minutes = 15×60

= 900 seconds

Total packets analyzed:

= $10,000 \times 900$

= 9,000,000 packets

Final Answer:

Total packets analyzed = 9,000,000 packets

Q18

Given:

Total packets analyzed = 200,000

Alerts generated = 5,000

Alert percentage:

= $5,000 / 200,000 \times 100$

= 2.5%

Final Answer:

Alert percentage = 2.5%

Q28

Given:

Total login attempts = 25,000

Failed login attempts = 1,250

Login failure percentage:

= $1,250 / 25,000 \times 100$

= 5%

Final Answer:

Login failure percentage = 5%

Possible brute-force sign:

Repeated login failures from the same IP address or same account may indicate a brute-force attack.

Result

Thus, Snort IDS packet monitoring and alert calculations were performed successfully.

EX: 09 - Implement Firewall Rules Using Snort

Aim

To understand how firewall or IDS rules control network traffic and calculate blocked IP count and throughput.

How to Use the Tool

1. Open the Snort rules folder:

```
C:\Snort\rules
```

2. Open the file:

```
local.rules
```

3. Add or study a custom Snort rule according to the lab requirement.
4. Open the Snort configuration file:

```
C:\Snort\etc\snort.conf
```

5. Ensure that local.rules is included in the configuration file.
6. Open **Command Prompt as Administrator** and move to:

```
cd C:\Snort\bin
```

7. Test the configuration:

```
snort -T -c C:\Snort\etc\snort.conf
```

8. Run Snort and observe alerts:

```
snort -A console -q -c C:\Snort\etc\snort.conf -i 1
```

9. Record the alert output and perform the required calculations.

Procedure

1. Study the concept of firewall rules and IDS rules. Understand how rules are used to allow, block, or alert network traffic.
2. Open the Snort rule file such as local.rules. Prepare or study a custom rule based on IP address, port number, protocol, or packet content.
3. Include the custom rule file in the Snort configuration file so that Snort can load and apply the rule during monitoring.
4. Identify the network range and blocked IP range given in the question. These values are used to calculate the number of blocked IP addresses.
5. Calculate the number of blocked IPs by subtracting the starting IP value from the ending IP value and adding one.

6. Study how rules help in controlling traffic by allowing trusted traffic, blocking unwanted traffic, and generating alerts for suspicious traffic.
7. Use the given data transfer and time values to calculate average network throughput in Mbps.
8. Record the blocked IP count, rule usage, throughput calculation, and final result.

Program / Calculation

Q9

Given:

Network = 192.168.10.0/24

Blocked range = 192.168.10.50 to 192.168.10.120

Number of blocked IPs:

= Last IP – First IP + 1

= 120 – 50 + 1

= 71

Final Answer:

Number of blocked IP addresses = 71

Q19

Rules help in controlling network traffic by:

Allowing trusted traffic

Blocking unwanted traffic

Detecting suspicious packets

Checking IP address

Checking port number

Checking protocol

Generating alerts

Enforcing security policy

Final Answer:

Rules help to allow safe traffic and block or alert suspicious traffic.

Q29

Given:

Data transferred = 900 MB

Time = 3 minutes

Convert time:

3 minutes = 180 seconds

Convert data:

$$\begin{aligned} 900 \text{ MB} &= 900 \times 8 \\ &= 7200 \text{ megabits} \end{aligned}$$

Average throughput:

$$\begin{aligned} &= 7200 / 180 \\ &= 40 \text{ Mbps} \end{aligned}$$

Final Answer:

Average throughput = 40 Mbps

Network performance:

40 Mbps is suitable for normal browsing, streaming, and file transfer.

Result

Thus, firewall rule calculation and traffic throughput were studied successfully.

EX: 10 - Generate Network Attack and Detect Using Snort

Aim

To monitor network traffic using Snort and calculate detection performance and critical alert percentage.

How to Use the Tool

1. Configure Snort in a controlled lab system.
2. Open **Command Prompt as Administrator**.
3. Move to the Snort binary folder:

```
cd C:\Snort\bin
```

4. Test the Snort configuration:

```
snort -T -c C:\Snort\etc\snort.conf
```

5. Start Snort monitoring on the selected interface:

```
snort -A console -q -c C:\Snort\etc\snort.conf -i 1
```

6. Use only safe lab traffic, sample packets, or given log values for detection practice.
7. Observe Snort console alerts or log files.
8. Record the number of logs, alerts, critical alerts, and detection performance values.
9. Perform the required calculation such as critical alert percentage.

Procedure

1. Configure Snort IDS in a controlled lab environment with proper rule files and logging path.
2. Ensure that the monitoring system and target system are placed in the same isolated lab network or virtual network.
3. Enable required Snort detection rules for identifying suspicious traffic and possible intrusion signatures.
4. Start Snort in monitoring mode and observe live network traffic. Snort compares packets with enabled rules and generates alerts when suspicious patterns are found.
5. Use only safe lab traffic, sample traffic, or log data for detection practice. Do not test attacks on public systems or unauthorized networks.
6. Observe whether Snort identifies suspicious packets and generates alerts in the console or log file.
7. Record the number of logs, alerts, critical alerts, or detection performance values given in the question.

8. Calculate detection improvement, processing impact, or critical alert percentage as required.
9. Save or note the Snort alert output and record the final result.

Program / Calculation

Q10

Given:

Detection accuracy improves by 10%

Processing time doubles

Effect:

Detection accuracy increases.

Processing time becomes two times higher.

System becomes slower.

More CPU and memory may be required.

Final Answer:

Security detection improves, but system performance decreases because processing time doubles.

Q20

Steps to monitor traffic using Snort:

Configure Snort interface

Enable required rules

Start packet monitoring

Observe network traffic

Check alerts and logs

Identify intrusion signatures

Tune rules if needed

Final Answer:

Snort monitors packets and compares them with known attack signatures. If suspicious traffic is found, it generates alerts.

Q30

Given:

Total logs per hour = 15,000

Critical alerts = 450

Critical alert percentage:

$= 450 / 15,000 \times 100$

$= 3\%$

Final Answer:

Critical alert percentage = 3%

Incident severity:

450 critical alerts per hour is serious and needs immediate investigation.

Result

Thus, network attack detection using Snort was studied successfully.